



Fullstack Web Development

Session 2 | Week 2/Day 2 | 120 min

CSS - Part02

Instructor: Ms. Liev Nova



090667393



CSS Position

What is CSS Position?

The **position** property in CSS determines **how an element is placed** on the page.

It affects:

- Layout flow
- Overlapping elements
- Scroll behavior

CSS Position

Position Types

CSS provides five main position values:

Position	Description
static	Default position; follows normal document flow
relative	Positioned relative to its normal position
absolute	Positioned relative to its nearest positioned ancestor
fixed	Positioned relative to the viewport ; does not move when scrolling
sticky	Switches between relative and fixed depending on scroll

CSS Position

Position Types

CSS provides five main position values:

- **Example:** `static` (Default)

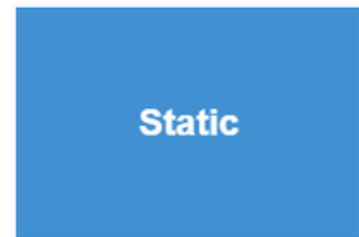
```
.static {  
  position: static;  
  background-color: #3498db;  
}
```

CSS

```
<div class="box static">Static</div>
```

HTML

Display



- Default behavior
- Element flows **normally in the page**
- **Top, right, bottom, left** properties **do not work**

CSS Position

Position Types

CSS provides five main position values:

- **Example:** *relative*

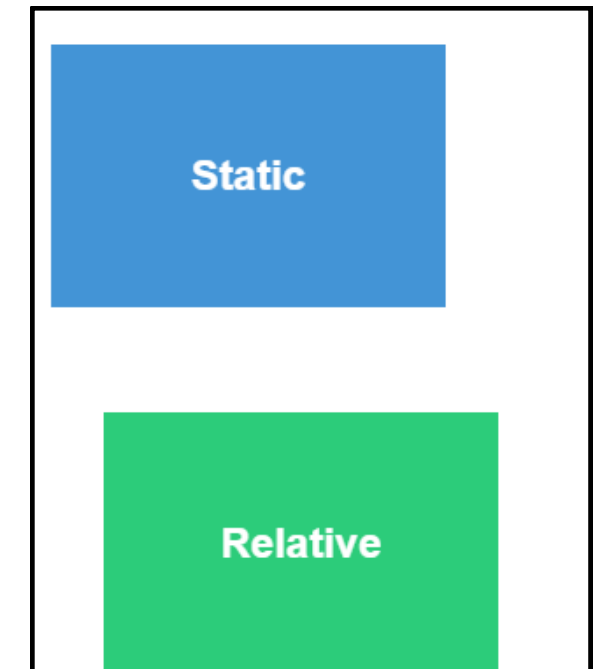
HTML

```
<div class="box relative">Relative</div>
```

CSS

```
.relative {  
  position: relative;  
  top: 20px;  
  left: 20px;  
  background-color: #2ecc71;  
}
```

Display



- Moves element **relative to its normal position**
- Space for the element **remains in the layout**
- Useful for **offsetting elements slightly**

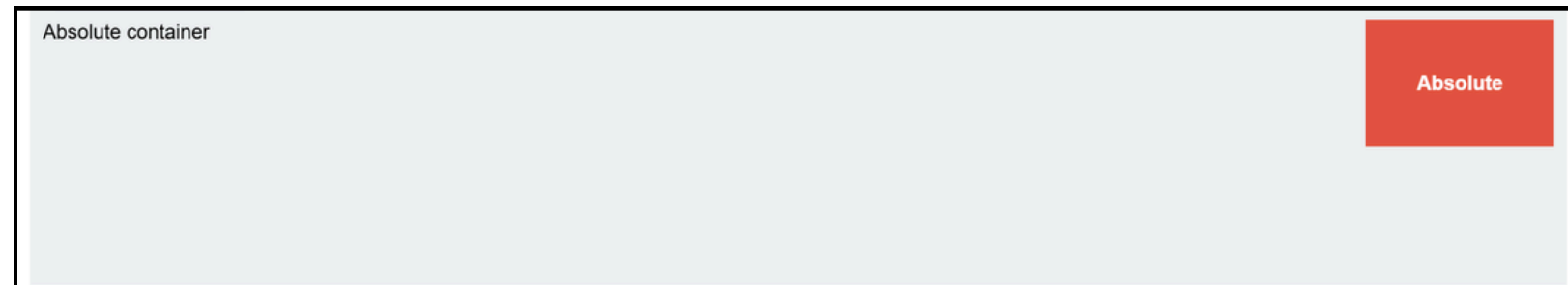
CSS Position

Position Types

CSS provides five main position values:

- **Example:** `absolute`

Display



HTML

```
<div class="absolute-container">
  <div class="box absolute">Absolute</div>
  Absolute container
</div>
```

CSS

```
.absolute-container {
  position: relative;
  height: 200px;
  background-color: #ecf0f1;
  margin-bottom: 20px;
  padding: 10px;
}

.absolute {
  position: absolute;
  top: 10px;
  right: 10px;
  background-color: #e74c3c;
}
```

- Positioned **relative to its nearest ancestor with position other than static**
- **Removed from normal flow**
- Can overlap other elements

CSS Position

Position Types

CSS provides five main position values:

- **Example:** `fixed`

Display



- Positioned **relative to the viewport**
- Stays in **same place when scrolling**
- Commonly used for **headers, footers, or floating buttons**

```
.fixed {  
  position: fixed;  
  top: 20px;  
  right: 20px;  
  background-color: #9b59b6;  
}
```

CSS

```
<div class="box fixed">Fixed</div>
```

HTML

CSS Position

Position Types

CSS provides five main position values:

- **Example:** sticky

```
<div class="sticky-container">
  <div class="box sticky">Sticky</div>
  Scroll inside this yellow container to see sticky in action.
  <p style="margin-top:150px;">More content...</p>
  <p>Even more content...</p>
</div>

<p>Scroll down the page to see the fixed element stay visible.</p>
```

HTML

```
.sticky-container {
  height: 300px;
  background-color: #f1c40f;
  overflow: auto;
  margin-bottom: 20px;
}

.sticky {
  position: sticky;
  top: 0;
  background-color: #e67e22;
}
```

CSS

CSS Position

Position Types

CSS provides five main position values:

- **Example:** sticky

Display



- Behaves like **relative** until scroll reaches a **threshold**
- Then **sticks to the viewport**
- Useful for **sticky headers** or **table headings**

CSS Float

What is Float?

The **float** property in CSS is used to **move elements to the left or right** of its container.

- Other content flows around it
- Commonly used for images and layouts

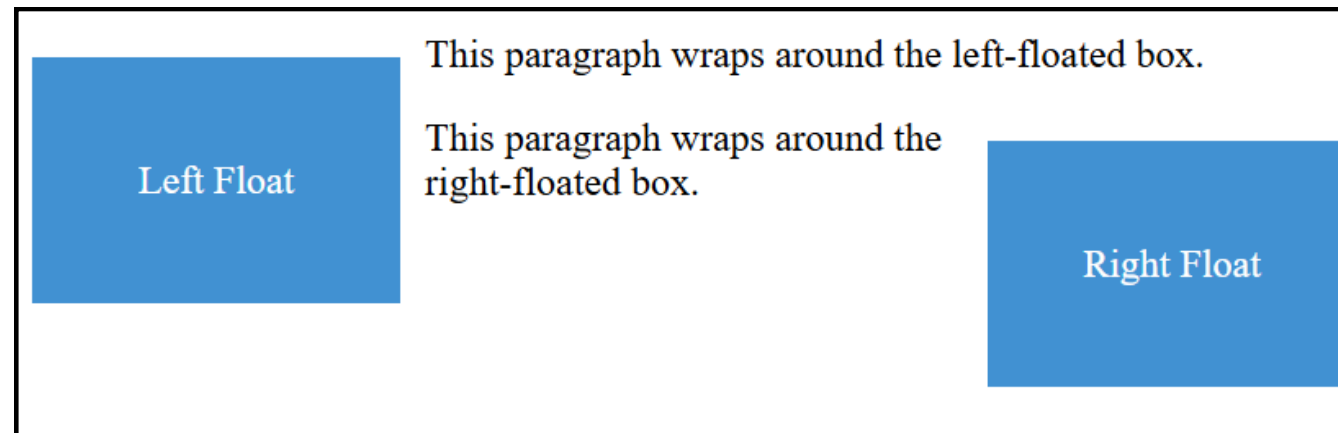
Values:

Value	Description
left	Float element to the left
right	Float element to the right
none	Default, element does not float
inherit	Inherit float from parent

CSS Position

- Float Example

Display



HTML

```
<div class="box float-left">Left Float</div>
<p>This paragraph wraps around the left-floated box.</p>

<div class="box float-right">Right Float</div>
<p>This paragraph wraps around the right-floated box.</p>
```

CSS

```
.box {
  width: 150px;
  height: 100px;
  background-color: #3498db;
  color: white;
  text-align: center;
  line-height: 100px;
  margin: 10px;
}

.float-left {
  float: left;
}

.float-right {
  float: right;
}
```

CSS Clear

What is Clear?

The **clear** property prevents elements from **wrapping around floated elements**.

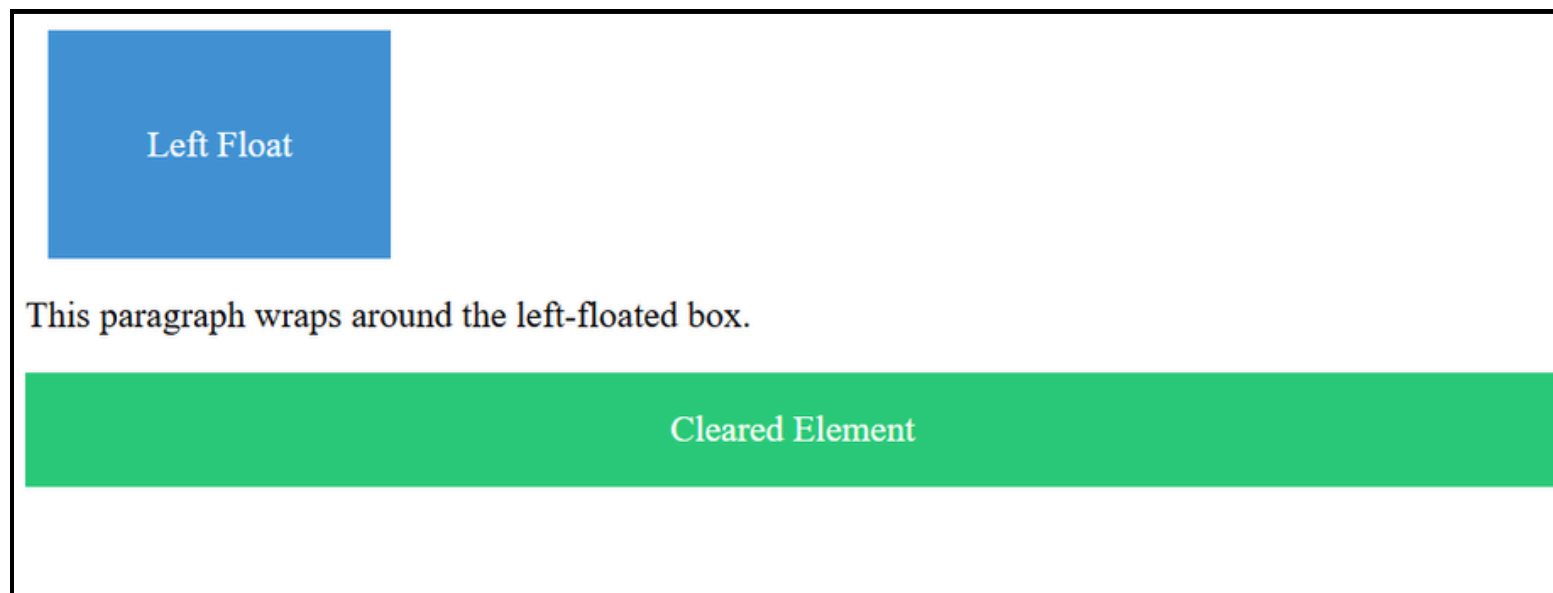
Values:

Value	Description
none	Default, element may appear next to floats
left	Element cannot be next to left-floated elements
right	Element cannot be next to right-floated elements
both	Element cannot be next to any floated elements
inherit	Inherit from parent

CSS Position

- Clear Example

Display



HTML

```
<div class="box float-left">Left Float</div>
<p>This paragraph wraps around the left-floated box.</p>

<div class="clear-both">Cleared Element</div>
```

CSS

```
.clear-both {
  clear: both;
  background-color: #2ecc71;
  height: 50px;
  text-align: center;
  line-height: 50px;
  color: white;
}
```


CSS Box Model

What is the CSS Box Model?

In CSS, the **Box Model** is used to describe the **layout and spacing of HTML elements**.

This box controls:

- Element size
- Spacing
- Borders
- Layout structure

CSS Box Model

Structure of the Box Model

The CSS Box Model consists of **four main parts**:

- Content
- Padding
- Border
- Margin

Each part affects **spacing and layout** on a webpage.

CSS Box Model

Box Model Layers

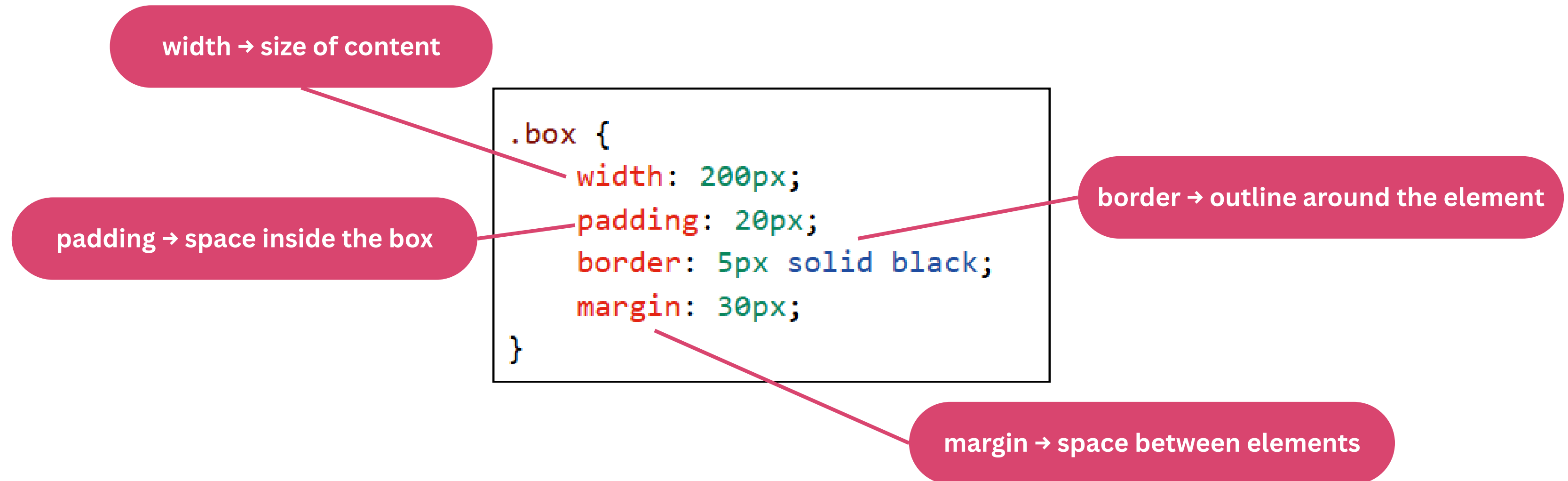
From **inside to outside**:

Part	Description
Content	Area where text, images, or other elements appear
Padding	Space between content and border
Border	Line surrounding padding and content
Margin	Space outside the border

CSS Box Model

Box Model Layers

Example:

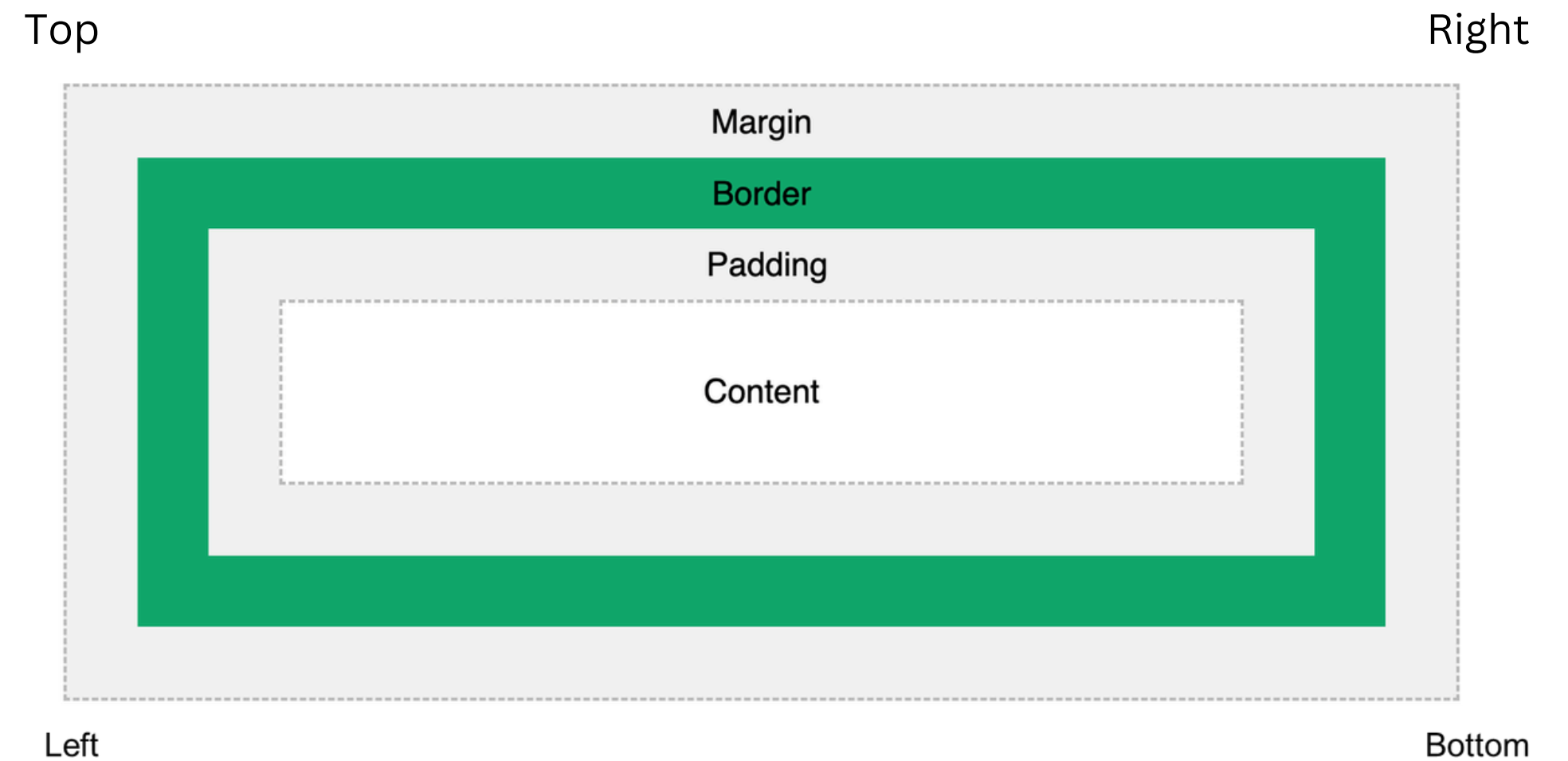


CSS Box Model

Box Model Layers

Example:

- Padding = space inside the box
- Margin = space outside the box



CSS Margins

What is Margin?

Margins are used to create **space around elements**, outside of the element's border.

Example uses:

- Space between paragraphs
- Space between sections
- Centering elements

CSS Margins

Margin Properties

CSS provides several margin properties

Property	Description
margin	Sets all margin sides in one declaration
margin-top	Sets the top margin
margin-right	Sets the right margin
margin-bottom	Sets the bottom margin
margin-left	Sets the left margin

Example:

```
div {  
  margin-top: 20px;  
  margin-right: 30px;  
  margin-bottom: 20px;  
  margin-left: 30px;  
}
```

CSS Margins

margin Shorthand Property

The `margin` property allows setting all margins in **one line**.

Example:

```
div {  
  margin: 20px 30px 20px 30px;  
}
```

Order:

- Top
- Right
- Bottom
- Left

CSS Margins

Margin Value Types

Margins can have different value types:

Value	Description
auto	Browser automatically calculates the margin
length	Fixed value such as px, pt, cm
%	Percentage based on container width
inherit	Inherit margin from parent element

Example:

```
.box1 {  
    margin: 40px;  
}  
  
.box2 {  
    margin-left: 50px;  
}  
  
.box3 {  
    margin: auto;  
}  
  
.box4 {  
    margin: 10%;  
}
```

CSS Padding

What is Padding?

Padding is used to create **space around an element's content**, inside the border.

Padding controls the **space between the content and the border** of an element.

Example uses:

- Creating space inside buttons
- Improving text readability
- Adjusting element spacing inside containers

CSS Padding

Padding Properties

CSS provides several padding properties:

Value	Description
padding	Sets all padding sides in one declaration
padding-top	Sets the top padding
padding-right	Sets the right padding
padding-bottom	Sets the bottom padding
padding-left	Sets the left padding

Example:

```
div {  
  padding-top: 20px;  
  padding-right: 30px;  
  padding-bottom: 20px;  
  padding-left: 30px;  
}
```


CSS Padding

padding Shorthand Property

The **margin** property allows setting all margins in **one line**.

Example:

```
div {  
  padding: 20px 30px 20px 30px;  
}
```

Order:

- Top
- Right
- Bottom
- Left

CSS Padding

Padding Value Types

Padding can use different value types:

Value	Description
length	Fixed size such as px, pt, cm
%	Percentage of the container width
inherit	Inherit padding from parent element

Default value: **0px**

Example:

```
div {  
    padding: 20px;  
}
```

CSS Tables

What is CSS Table Styling?

CSS allows us to **style HTML tables** to improve appearance and readability.

Using CSS we can control:

- Table borders
- Table size
- Text alignment
- Cell spacing
- Row highlighting
- Responsive tables

CSS Tables

Table Border : **border** Property

The **border** property is used to define **table borders**.

Example:

```
table, th, td {  
    border: 1px solid black;  
}
```

This adds **borders** around:

- the table
- table headers **<th>**
- table cells **<td>**

CSS Tables

Border Collapse

The `border-collapse` property controls how table borders appear.

Example:

```
table {  
  border-collapse: collapse;  
}
```

Effect:

- Combines multiple borders into one **single border**
- Makes the table cleaner and easier to read

CSS Tables

Text Alignment : `text-align`

The `text-align` property controls **horizontal alignment** of text in table cells.

Values:

- left
- center
- right

Example:

```
th {  
    text-align: center;  
}
```

CSS Tables

Vertical Alignment : `vertical-align`

The `vertical-align` property controls **vertical positioning** of content.

Values:

- top
- middle
- bottom

Example:

```
td {  
    vertical-align: middle;  
}
```

CSS Tables

Table Padding: padding

Padding creates **space inside table cells**.

Example:

```
th, td {  
    padding: 10px;  
}
```

Effect:

- Improves table readability
- Adds space between cell content and border

CSS Tables

Horizontal Dividers: `border-bottom`

Used to create **horizontal lines between rows**.

Example:

```
th, td {  
    border-bottom: 1px solid #ddd;  
}
```

CSS Tables

Hover Effect: `tr:hover`

The `:hover` selector highlights rows when the mouse is over them.

Example:

```
tr:hover {  
    background-color: #f2f2f2;  
}
```

This improves **user interaction and readability**.

CSS Tables

Responsive Tables: `overflow-x: auto`

To make tables responsive on small screens, wrap them inside a container.

Example:

```
.table-container {  
    overflow-x: auto;  
}
```

CSS Flexbox

What is Flexbox?

Flexbox (Flexible Box Layout) is a CSS layout model used to create **flexible and responsive layouts**.

It helps developers arrange elements **easily without using float or complex positioning**.

Benefits:

- Easy alignment
- Flexible layout control
- Responsive design
- Less CSS code

CSS Flexbox

Flexbox Layout : Flexbox Behavior

When a container uses **Flexbox**, its items are arranged **in a row by default**.

Default direction:

- Horizontal layout

Example layout:



CSS Flexbox

Flexbox Structure

Flexbox has two main components:

Element	Description
Flex Container	The parent element that enables flexbox
Flex Items	The child elements inside the container

HTML

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>
```

```
.flex-container {
  display: flex;
  background-color: lightblue;
}

.flex-container div {
  background-color: white;
  margin: 10px;
  padding: 20px;
  font-size: 20px;
}
```

CSS

CSS Flexbox

Flexbox Main Axis — Flex Direction

The **flex-direction** property defines **the direction of flex items inside a flex container**.

It also determines the **main axis** of the layout.

The **main axis** is the **primary direction in which flex items are placed**.

CSS Flexbox

Flex Direction Values — `flex-direction` Values

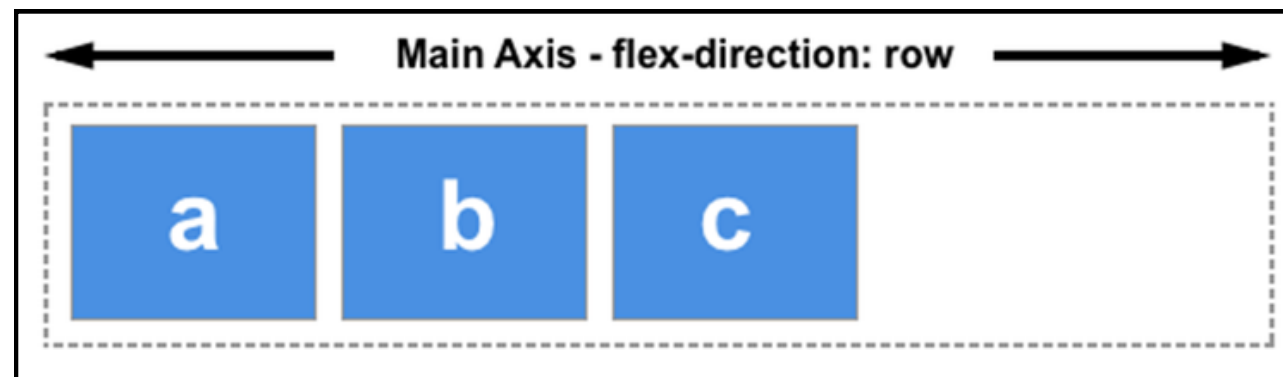
Value	Description
Value	Description
row	Items are arranged horizontally (left → right)
row-reverse	Items are arranged horizontally (right → left)
column	Items are arranged vertically (top → bottom)
column-reverse	Items are arranged vertically (bottom → top)

CSS Flexbox

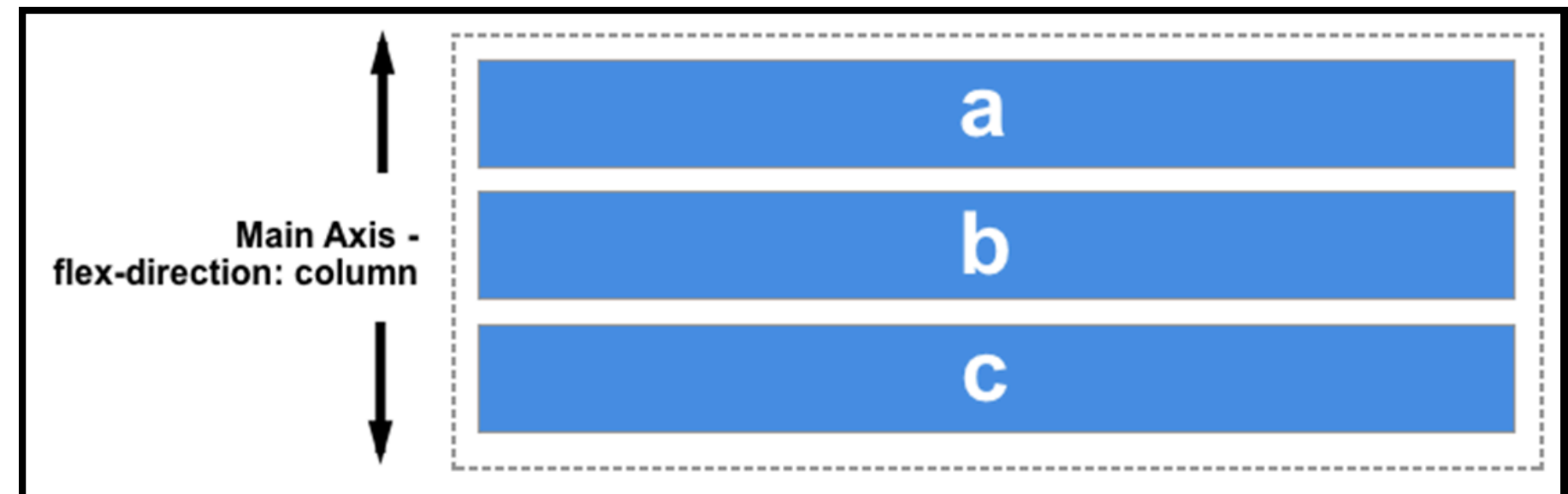
Main Axis — Main Axis Direction

The **main axis** changes depending on the **flex-direction** value.

1. flex-direction: row



2. flex-direction: column



CSS Flexbox

What is the Cross Axis?

In **Flexbox**, there are two axes:

- **Main Axis** → controlled by flex-direction
- **Cross Axis** → runs perpendicular to the main axis

Example: **Axis Visualization — Axis Direction**

If **flex-direction: row**

- Main Axis → horizontal
- Cross Axis → vertical

If **flex-direction: column**

- Main Axis → vertical
- Cross Axis → horizontal

CSS Flexbox

Aligning Items on the Cross Axis — `align-items` Property

The `align-items` property is used to **align flex items along the cross axis**.

Syntax :

```
.container{  
  display:flex;  
  align-items:value;  
}
```

CSS Flexbox

Aligning Items on the Cross Axis — `align-items` Values

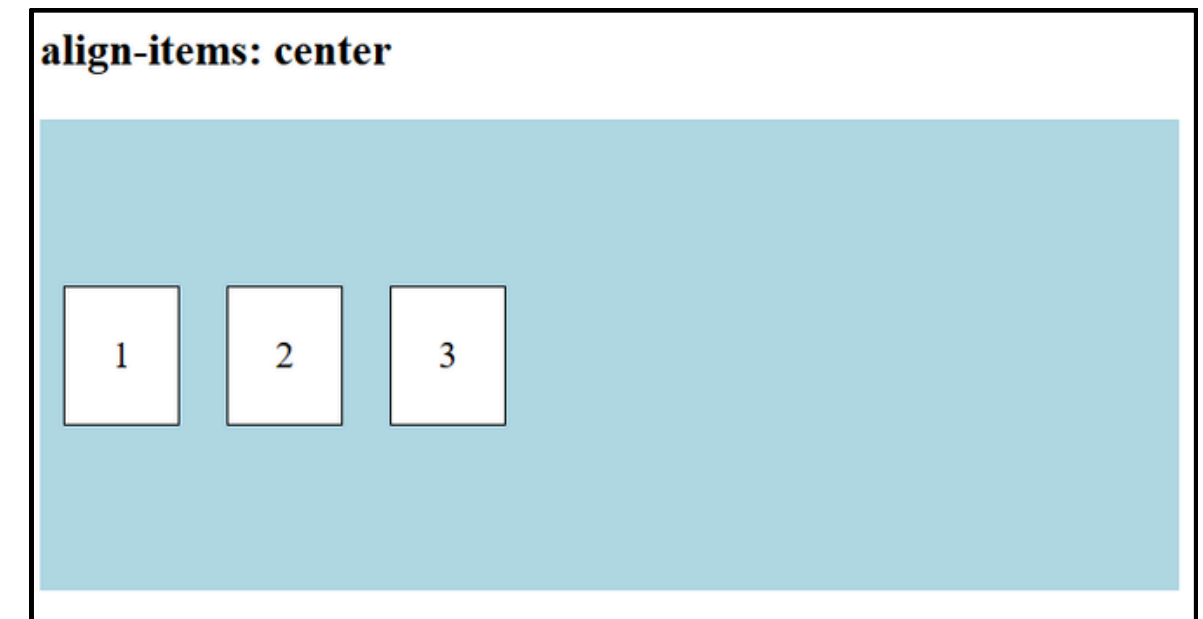
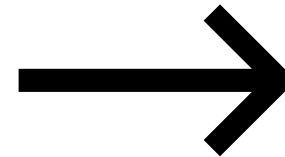
Value	Description
flex-start	Items align at the start of the cross axis
flex-end	Items align at the end of the cross axis
center	Items align at the center
baseline	Items align based on text baseline
stretch	Items stretch to fill the container (default)

CSS Flexbox

Aligning Items on the Cross Axis — `align-items` Property

Example :

```
.container {  
  display: flex;  
  height: 200px;  
  background-color: lightblue;  
  align-items: center;  
}
```



CSS Flex Container

What is a Flex Container?

A **Flex Container** is the parent element that uses **Flexbox layout**.

To create a flex container, use:

```
display: flex;
```

- Example:

```
<div class="container">  
  <div>1</div>  
  <div>2</div>  
  <div>3</div>  
</div>
```

All direct child elements automatically become flex items.

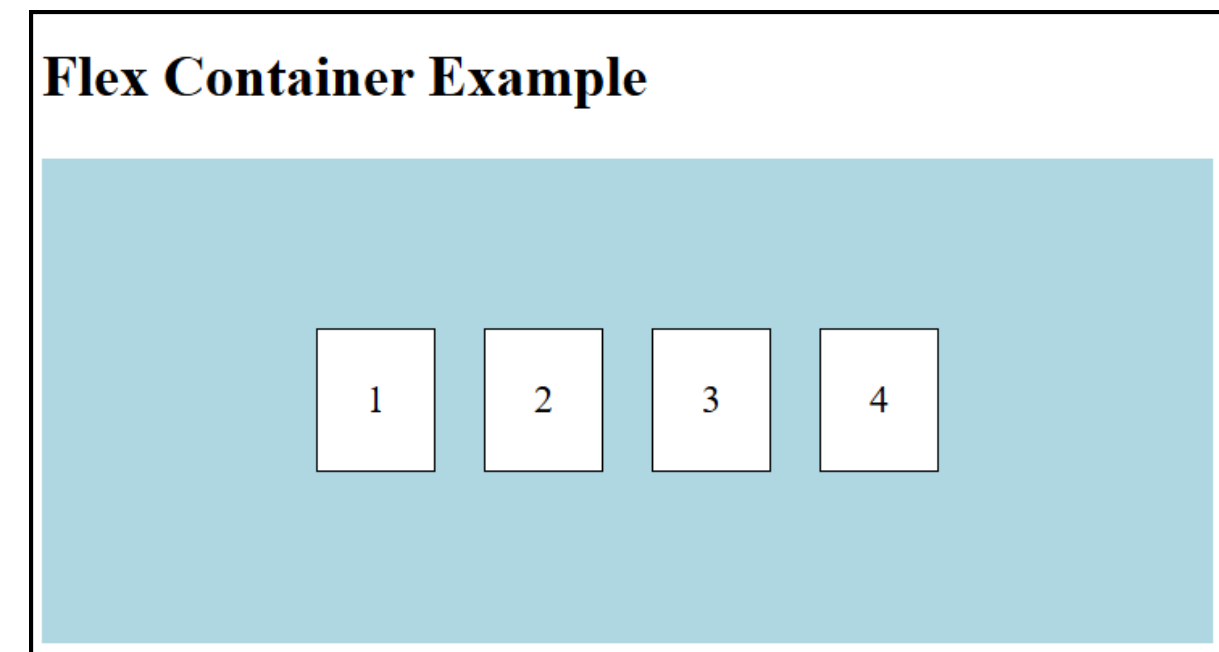
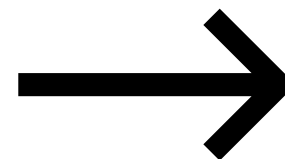
CSS Flex Container

What is a Flex Container?

A **Flex Container** is the parent element that uses **Flexbox layout**.

Example:

```
.container {  
  display: flex;  
  flex-direction: row;  
  flex-wrap: wrap;  
  justify-content: center;  
  align-items: center;  
  height: 200px;  
  background-color: lightblue;  
}  
  
.container div {  
  background: white;  
  margin: 10px;  
  padding: 20px;  
  border: 1px solid black;  
}
```



CSS Flex Container

Main Flex Container Properties

CSS provides several properties to control the behavior of a flex container.

Property	Purpose
display	Defines the element as a flex container
flex-direction	Defines the direction of flex items
flex-wrap	Allows items to wrap onto multiple lines
flex-flow	Shorthand for flex-direction and flex-wrap
justify-content	Aligns items along the main axis
align-items	Aligns items along the cross axis
align-content	Aligns multiple flex lines

CSS Flex Container

Main Flex Container Properties

Display Property — `display`

The display property defines the type of layout used.

Example:

```
.container{  
    display: flex;  
}
```

This turns the element into a **flex container**.

CSS Flex Container

Main Flex Container Properties

Display Property — **flex-direction**
Defines the direction of **flex items**.

Values:

- row
- row-reverse
- column
- column-reverse

- Example:

```
.container{  
    flex-direction: row;  
}
```

CSS Flex Container

Main Flex Container Properties

Display Property — flex-wrap

The **flex-wrap** property controls whether items wrap to a new line.

Values:

- nowrap (default)
- wrap
- wrap-reverse

- Example:

```
.container{  
    flex-wrap: wrap;  
}
```

CSS Flex Container

Main Flex Container Properties

Display Property — flex-flow

The flex-flow property is shorthand for:

- flex-direction
- flex-wrap

This means:

- direction → row
- wrapping → wrap

- Example:

```
.container{  
    flex-flow: row wrap;  
}
```

CSS Flex Container

Main Flex Container Properties

Display Property — justify-content

The **justify-content** property aligns items along the **main axis**.

Values include:

- flex-start
- flex-end
- center
- space-between
- space-around
- space-evenly

- Example:

```
.container{  
    justify-content: center;  
}
```

Items will be centered horizontally.

CSS Flex Container

Main Flex Container Properties

Display Property — align-items

The **align-items** property aligns items along the **main axis**.

Values:

- flex-start
- flex-end
- center
- baseline
- stretch

- Example:

```
.container{  
    align-items: center;  
}
```

Items will be centered vertically.

CSS Flex Container

Main Flex Container Properties

Display Property — align-content

The **align-content** property aligns **flex lines**, not individual items.

This works only when there are multiple lines of flex items.

Example:

```
.container{  
    align-content: space-between;  
}
```

CSS Flex Container

Display Property — align-content

The **align-content** property aligns **flex lines**, not individual items.

Values:

Value	Description
stretch	Stretches rows to fill the container
flex-start	Packs rows at the top
flex-end	Packs rows at the bottom
center	Packs rows in the center
space-between	Even space between rows
space-around	Equal space around rows
space-evenly	Equal space between and around rows

CSS Flex Items

What are Flex Items?

In **Flexbox**, the **child elements inside a flex container** are called **flex items**.

Example:

```
<div class="container">  
  <div>1</div>  
  <div>2</div>  
  <div>3</div>  
  <div>4</div>  
</div>
```

- container → Flex Container
- 1, 2, 3, 4 → Flex Items

CSS Flex Items

Flex Item Properties — Main Flex Item Properties — **order**

The **order** property changes the **display order of flex items**.

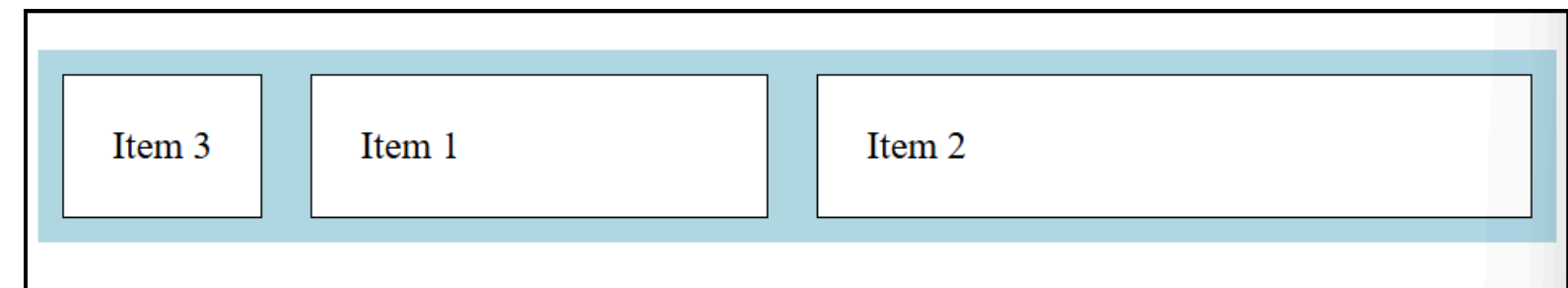
Example

```
<div class="container">
  <div class="item1">Item 1</div>
  <div class="item2">Item 2</div>
  <div class="item3">Item 3</div>
</div>
```

```
.item1 {
  flex-grow: 1;
}

.item2 {
  flex-grow: 2;
}

.item3 {
  order: -1;
}
```



Result: Items will appear in order: 3 → 1 → 2

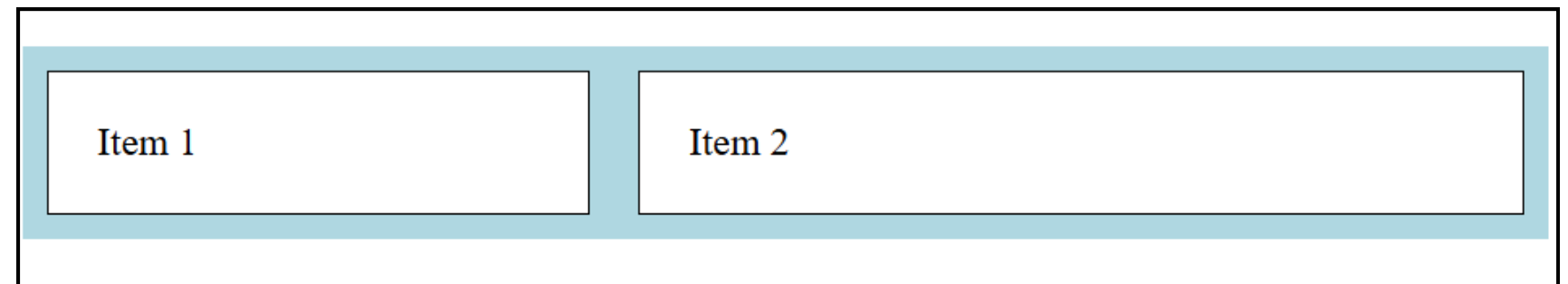
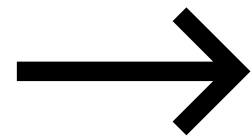
CSS Flex Items

Flex Item Properties — Main Flex Item Properties — `flex-grow`

The `flex-grow` property defines how much a flex item **grows compared to other items**.

Example

```
.item1{  
  flex-grow:1;  
}  
  
.item2{  
  flex-grow:2;  
}
```



Result: Item 2 grows twice as much as item 1.

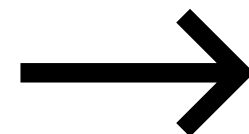
CSS Flex Items

Flex Item Properties — Main Flex Item Properties — `flex-shrink`

The `flex-shrink` property controls how much a flex item **shrinks when space is limited**.

Example

```
.item1{  
  flex-shrink:1;  
}  
  
.item2{  
  flex-shrink:2;  
}
```



Result: Item 2 shrinks twice as much as item 1.

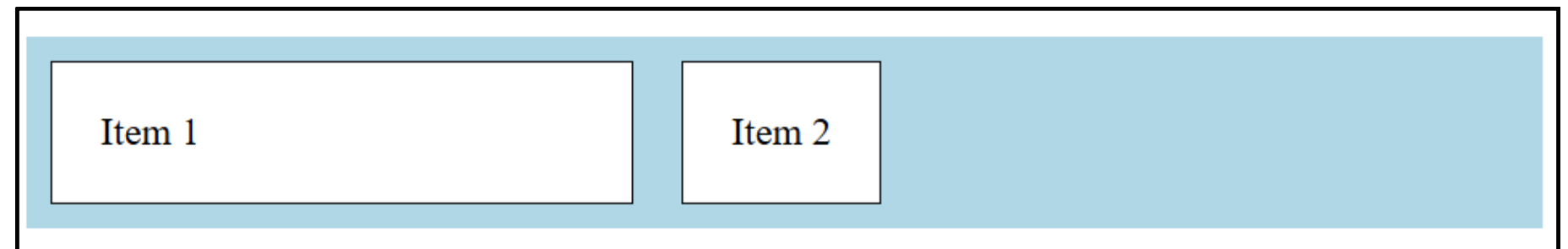
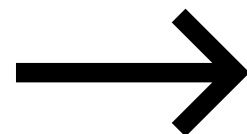
CSS Flex Items

Flex Item Properties — Main Flex Item Properties — *flex-basis*

The *flex-basis* property defines the **initial size of a flex item**.

Example

```
.item1{  
  flex-basis:200px;  
}
```



Result: The item starts with a 200px width before growing or shrinking.

CSS Flex Items

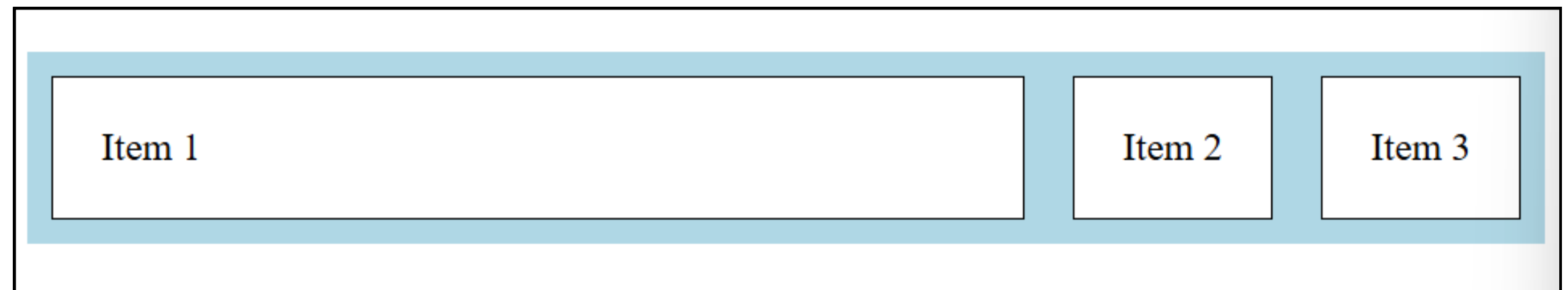
Flex Item Properties — Main Flex Item Properties — flex

The **flex** property combines:

- flex-grow
- flex-shrink
- flex-basis

Example

```
.item1{  
  flex: 1 1 200px;  
}
```



Values include: grow → 1 , shrink → 1 and basis → 200px

CSS Flex Items

Flex Item Properties — Main Flex Item Properties — `align-self`

The `align-self` property overrides the container's **`align-items`** value for a specific item.

Example

```
.item2{  
  align-self: center;  
}
```

Values include: flex-start , flex-end , center , stretch and baseline

CSS Grid

What is CSS Grid?

CSS Grid Layout is a **two-dimensional layout system** used to design web pages. It allows you to arrange elements in **rows and columns**.

Benefits of CSS Grid:

- Create complex layouts easily
- Control both **rows and columns**
- Build **responsive layouts**

CSS Grid

Grid Container and Grid Items

To use Grid, you must define a **grid container**.

Example:

```
<div class="grid-container">  
  <div>1</div>  
  <div>2</div>  
  <div>3</div>  
  <div>4</div>  
</div>
```

- **Grid Container** → parent element
- **Grid Items** → elements inside the container

CSS Grid

Enable Grid Layout

To activate Grid layout, use:

Example:

```
.grid-container{  
  display: grid;  
}
```

- This converts the container into a grid layout system.

CSS Grid

Define Columns

Use `grid-template-columns` to define column sizes.

Example:

```
.grid-container{  
  display: grid;  
  grid-template-columns: auto auto auto;  
}
```

- This creates 3 columns.

CSS Grid

Define Rows

Use **grid-template-rows** to define row height.

Example:

```
.grid-container{  
  display: grid;  
  grid-template-columns: auto auto auto;  
  grid-template-rows: 100px 100px;  
}
```

- This creates 2 rows with height 100px each.

CSS Grid

Grid Gap

Use **gap** creates spacing between grid items.

Example:

```
.grid-container{  
  display: grid;  
  grid-template-columns: auto auto auto;  
  gap: 10px;  
}
```

- Result: 10px spacing between grid cells.

CSS Grid

Common Grid Properties

Property	Description
display: grid	Enables grid layout
grid-template-columns	Defines number and size of
grid-template-rows	Defines row sizes
gap	Space between rows and
grid-column	Control item width across
grid-row	Control item height across rows

CSS Flexbox

Flexbox vs. Grid

CSS Flexbox is used for a one-dimensional layout, with rows OR columns.

CSS Grid is used for a two-dimensional layout, with rows AND columns.



CSS Media Queries

What is a Media Query?

A **Media Query** allows CSS to apply **different styles for different devices or screen sizes**.

It is used to create **responsive web design**.

Responsive design means:

- Websites work on **mobile**
- Websites work on **tablets**
- Websites work on **desktops**

CSS Media Queries

Mobile First Design — Mobile First Approach

When using media queries, it is recommended to **design for mobile first**.

Steps:

1. Write CSS for **small screens**
2. Add media queries for **larger screens**

Benefits:

- Better performance
- Cleaner responsive design
- Easier scaling

CSS Media Queries

What Media Queries Can — Check Media Query Conditions

Media queries can check:

- Media queries can check:
- Viewport width
- Viewport height
- Device width
- Device height
- Screen orientation
- Screen resolution

Example:

```
@media (max-width: 600px)
```

CSS Media Queries

What Media Queries Can — Check Media Query Conditions

Media queries can check:

- Media queries can check:
- Viewport width
- Viewport height
- Device width
- Device height
- Screen orientation
- Screen resolution

Example:

```
@media (max-width: 600px)
```

CSS Media Queries

Example — Responsive Layout — Media Query Example

```
.container{  
    background-color: lightgray;  
}  
  
@media (max-width: 600px){  
    .container{  
        background-color: lightgreen;  
    }  
}
```

- **Meaning:**
 - Large screen → gray background
 - Small screen → green background

CSS Media Query Breakpoints

What Are Breakpoints?

Breakpoints are **specific screen widths** where your CSS rules change to adapt the layout.

They are essential for **responsive web design**.

CSS Media Query Breakpoints

Common Breakpoints

Breakpoint	Description	Dimensions
X-Small	Phones	<576px
Small	Portrait tablets & large phones	≥576px
Medium	Landscape tablets	≥768px
Large	Laptops / desktops	≥992px
Extra Large	Large laptops / desktops	≥1200px
Extra Extra Large	Extra wide screens	≥1400px

CSS Media Query Breakpoints

Media Query Syntax for Breakpoints

Extra Small Devices (Phones, <576px)

```
@media only screen and (max-width: 576px) {  
    /* CSS rules here */  
}
```

Small Devices (≥576px)

```
@media only screen and (min-width: 576px) {  
    /* CSS rules here */  
}
```

CSS Variables

What Are CSS Variables?

CSS Variables (also called **Custom Properties**) allow you to **store values that can be reused** throughout your CSS.

Benefits:

- Maintain consistent design
- Easy to update colors, sizes, and other values
- Can be global or local
- Can be manipulated via JavaScript or media queries

CSS Variables

Syntax

Declare a Variable

```
:root {  
  --primary-bg-color: green;  
  /* global variable */  
}  
  
.note {  
  --note-bg: yellow;  
  /* local variable */  
}
```

- `--variable-name` → custom variable name
- Global variables declared in `:root` are **available throughout the document**
- Local variables declared inside a selector are **only available inside that selector**

CSS Variables

Use a Variable

Using `var()` Function

```
body {  
    background-color: var(--primary-bg-color);  
}  
  
.note {  
    background-color: var(--note-bg);  
}
```

- `var(--variable-name)` retrieves the value of the variable
- Makes **changing the theme or colors easy**

CSS Variables

Variables and Media Queries

CSS variables can change depending on **media queries**.

```
:root {  
  --primary-bg-color: green;  
}  
  
@media (max-width: 600px) {  
  :root {  
    --primary-bg-color: orange;  
    /* change for small screens */  
  }  
}  
  
body {  
  background-color: var(--primary-bg-color);  
}
```

On screens smaller than 600px, the background color changes automatically



THANK YOU

For your attention !

